

Answer Key — Constructors & the Order Things Get Built

Instructor reference for the review worksheet — page 1 of 2

part 1 What Is a Constructor? — Q1

1. `name`
2. `return`
3. `object`

part 2 Default Constructor — Q2, Q3

Q2: `hero.name = "Unknown"`, `hero.health = 100` (the default constructor runs).

Q3 model answer:

```
Weapon() : name("Fists"), damage(1) {}
```

part 3 Overloaded Constructors — Q4, Q5

STATEMENT	CONSTRUCTOR	NAME	HEALTH
<code>Player a;</code>	<code>Player()</code>	"Unknown"	100
<code>Player b("Kevin");</code>	<code>Player(string n)</code>	"Kevin"	100
<code>Player c("Terry", 9000);</code>	<code>Player(string n, int h)</code>	"Terry"	9000

Q5: C++ distinguishes overloads by parameter *types*, never parameter names. `Player(string playerName)` has the exact same signature as `Player(string n)` — one `string` parameter — so it's a duplicate declaration of the same constructor, not a new overload. This does not compile.

part 4 Copy Constructor — Q6, Q7

Q6: `original` → `name="Mario"`, `health=95`. `copy` → `name="Mario"`, `health=95`. Same object? **NO**. Same values? **YES**.

Q7: `Player b(a);` → **COPY** · `Player c = a;` → **COPY** (copy-initialization — `c` doesn't exist yet, so despite the `=` this calls the copy constructor, not assignment) · `Player d;` → **NOT COPY** (default ctor) · `Player e("Alice");` → **NOT COPY** (overloaded ctor).

Answer Key — Constructors & the Order Things Get Built

Instructor reference for the review worksheet — page 2 of 2

part 5 Why Copy Constructors Matter — Q8

Shallow-copying a pointer duplicates the *address*, not the memory it points to. Two objects end up owning the same heap block; when either is destroyed (or reassigned/freed), the other is left with a dangling pointer, and if both eventually free the block, that's a double free. Modifying data through one object also silently affects the other.

part 6 Initializer Lists — Q9

```
Weapon(string n, int d)
: name(n), damage(d)
{
}
```

part 9 Find the Error — Q12

Course NO, it will not compile — `number` is `const`, so it must receive its value before the constructor body executes; assigning to it in the body (`number = 2143;`) is illegal. Fix: `Course() : number(2143), name("OOP") {}`

part 10 Write the Class — Book (model answer)

```
class Book {
    string title;
    string author;
    int pages;
public:
    Book()
        : title("Unknown"), author("Unknown"), pages(0) {}
    Book(string t, string a)
        : title(t), author(a), pages(0) {}
    Book(string t, string a, int p)
        : title(t), author(a), pages(p) {}
    Book(const Book& other)
        : title(other.title), author(other.author), pages(other.pages) {}
};
```

part 8 / 11 Trace tables

ROBOT	CONSTRUCTOR	NAME	BATTERY
a	Robot()	"Unknown"	100
b	Robot(string n)	"R2D2"	100
c	Robot(string n, int b)	"Johnny 5"	75
d	Robot(const Robot&)	"Johnny 5"	75

BOOK	TITLE	AUTHOR	PAGES
a	"Unknown"	"Unknown"	0
b	"Dune"	"Frank Herbert"	0
c	"The Hobbit"	"J.R.R. Tolkien"	310
d	"The Hobbit"	"J.R.R. Tolkien"	310

part 12 Constructor Challenge

Output: **A, B, C, C**. Thing `d = c;` is the common surprise — even though it uses `=`, `d` doesn't exist yet, so this is copy-initialization and calls the copy constructor (prints `"C"`), the same lesson as Q7.